Kafka: Complete Reference

Author: Mohamed Fouad Fakhruldeen (Fakhr.me)

Date: Nov 9, 2026

Course Portal: fakhr.me/KafkaSpark

Code & Labs Repository: github.com/fakhruldeen/KafkaSpark

Scope and Sources

This reference covers every architectural concept across enterprise Apache Kafka: topics, segments, retention, compaction, brokers, replication, KRaft, producers, consumers, consumer groups, offsets, delivery semantics, rebalancing, administration, upgrades, security, monitoring, multi-cluster replication and advertised listeners.

Config defaults are given as standard Apache Kafka 3.x/4.x behavior; where versions differ (e.g. pre-3.0 vs 3.0+ defaults), the section explicitly flags it.

1. Introduction: What Kafka Is and Where It Fits

Apache Kafka is an open-source distributed event streaming platform. It stores streams of records in topics, and applications write to and read from those topics at their own pace. LinkedIn built it for internal stream processing that traditional message queues could not handle. It was first released in January 2011 and is now an Apache Software Foundation project.

A data stream is a potentially unbounded sequence of records, valuable because it can be read as soon as it is produced. Stream volumes vary widely: some sources emit tens of thousands of records per second, others one or two per hour.

Kafka solves an integration problem. Many systems (billing, CRM, websites, accounting) each speak different protocols and formats. Point-to-point links between them become fragile. With Kafka in the middle, sources publish once, and any number of targets read independently, so the systems stay decoupled.

Common Uses:

- Messaging between microservices
- Activity tracking, such as clicks and page views
- Metrics collection from many sources, including IoT devices
- Application log aggregation
- Decoupling system dependencies
- Feeding big-data tools such as Apache Spark, Flink, and Hadoop

- An event-sourcing immutable ledger
- The storage layer under stream processors such as Flink and Samza

Poor Fits:

- Serving millions of clients directly, such as mobile or browser apps (use API Gateways/WebSockets proxies instead).
- Database workloads that need indexes or ad hoc SQL queries (a topic is an append-only log with no query engine).
- Work queues where each message is deleted after one consumer handles it (Kafka keeps messages for the retention period, and consumer parallelism is capped by partition count).
- Embedded real-time systems on small devices.
- Blockchain (Kafka shares append-only logs and immutability, but has no cryptographic verification and does not keep full history by design).

Ecosystem Components:

- **Kafka Streams:** A lightweight client library for transforming topics (filter, windowed aggregates, joins) without hand-written producer and consumer loops.
- **Kafka Connect:** A distributed framework with reusable source connectors (Debezium CDC, JDBC, MongoDB, SQS) and sink connectors (Postgres, S3, Elasticsearch, Snowflake).
- **Schema Registry:** Central contract store for Avro, Protobuf, and JSON Schema that enforces compatibility rules (BACKWARD, FORWARD, FULL) across producer/consumer evolution.
- **ksqlDB:** An event streaming database with a SQL-like interface for continuous stream processing.

---

2. Core Concepts and Vocabulary

Term	Meaning
Record	A key (optional), value (optional), headers, and timestamp. Gets a partition and offset on write
Topic	A named stream of records, split into partitions
Partition	An ordered, append-only log; the fundamental unit of ordering, parallelism, and storage
Offset	A record's sequential position within one partition. Starts at 0, never reused
Broker	One Kafka server node. A cluster is a group of brokers
Bootstrap Server	Any broker a client connects to first; it returns the complete cluster metadata
Leader / Follower	Per partition, one replica leads (serves reads/writes); the others replicate asynchronously
ISR (In-Sync Replicas)	The dynamic pool consisting of the leader plus followers that are fully caught up
Replication Factor	Total number of physical copies of each partition across distinct brokers
Controller	The node managing cluster metadata and leader elections (KRaft quorum or legacy ZooKeeper)
Producer	A client application that serializes and writes records to topics
Consumer Group	A set of consumers sharing a `group.id` that divide a topic's partitions among themselves
Coordinator	The specific broker designated to manage a consumer group's membership and offset commits
Segment	A physical disk file (`.log`) holding a slice of a partition's log, paired with index files
Retention / Compaction	Cleanup policies: delete by age/size, or compact to keep the latest value per key
Idempotent Producer	A producer that never writes duplicate records upon internal network retries (`enable.idempotence=true`)
Transaction	An atomic group of writes and offset commits that commit or abort together across topics

3. Topics, Partitions and Offsets

A topic is a named, ordered stream. It can hold any message payload format (bytes).

Partitions

Each topic is split into numbered partitions (0 to $N-1$). A partition is the unit of **ordering, parallelism, and storage**.

- The partition count is set at creation. It can be increased later via `kafka-topics --alter` but **never decreased**.
- **Warning:** Adding partitions alters MurmurHash2 key routing; ordering for existing entity keys will break across partitions.

Ordering Guarantees

Scenario	Ordering Guarantee
Records with the **same key**	**Strictly ordered**, because they hash to the exact same partition
Records with **no key** (`null`)	Not globally ordered; distributed via sticky/round-robin batching
Records across **different partitions**	**No ordering guarantee**

Offsets

- Monotonically increasing 64-bit integer assigned by the broker upon disk append.
- Offsets start at 0 in each partition and are immutable.
- An offset is unique **only within its partition** (Partition 0 Offset 5 is unrelated to Partition 1 Offset 5).

Sizing Rules of Thumb

- **Throughput Formula:** $\text{Partitions} = \frac{\text{Target Throughput}}{\text{Single-Partition Consumer Throughput}}$
- A single broker practically handles 2,000 to 4,000 partitions.
- Small volume (<1 GB/day): 1 to 3 partitions.
- Medium volume (1–10 GB/day): 6 to 12 partitions.
- High volume (>10 GB/day): 20+ partitions.

Large Messages (>1 MB)

Kafka's default ceiling is 1 MB. To handle larger records:
1. **Compress first** (`compression.type=snappy` or `lz4`).
2. **Claim-Check Pattern:** Upload blob to S3/GCS/MinIO, produce only the payload URL + checksum to Kafka.
3. **Payload Chunking:** Split payload into sequence-numbered chunks.
4. If raising limits, configure all 4 tiers simultaneously:
- Producer: `max.request.size`
- Broker: `message.max.bytes`
- Topic: `max.message.bytes`
- Consumer: `max.partition.fetch.bytes` and `fetch.max.bytes`

---

4. Segments, Retention and Compaction

Segments and Physical Index Files

A partition directory on disk (`<topic>-<partition>`) contains segments:
- `.log` : Raw binary record batches.
- `.index` : Sparse memory-mapped index translating logical offsets to physical byte offsets.
- `.timeindex` : Maps message timestamps to offsets for point-in-time seeks.
- `leader-epoch-checkpoint` : Monotonic epoch counter preventing log truncation divergence in KRaft.
- A segment rolls (closes) when it hits `segment.bytes` (default 1 GB) or `segment.ms` (default 7 days). Only closed segments are compacted or deleted.

Retention Policies

- `cleanup.policy=delete` (default):
- `retention.ms` : Millisecond TTL (e.g. 86400000 = 24h, 604800000 = 7 days).
- `retention.bytes` : Maximum bytes retained per partition (-1 = unlimited).
- `cleanup.policy=compact` :
- Keeps the latest value for each key. Older duplicate keys in closed segments are purged.
- A record with a `null` value is a **Tombstone**, signaling key deletion.
- Tombstones are retained for `log.cleaner.delete.retention.ms` (default 24h) before physical deletion so downstream readers observe the deletion event.

---

5. Brokers, Clusters and Advertised Listeners

Network Listeners & Address Resolution

A classic failure in production is the misconfiguration of listeners:
- `listeners` : Network socket interfaces the broker binds to (e.g., `PLAINTEXT://0.0.0.0:9092`).
- `advertised.listeners` : Host/port returned to clients in the metadata response. Clients use these addresses for all subsequent produce/fetch requests!
- **Never advertise `localhost` or internal private IPs to external or remote clients.**

Replication, Durability & The High-Availability Quorum

Each partition has 1 Leader and $RF-1$ Followers.
- **`acks=0` :** Fire and forget. Highest throughput, zero durability.
- **`acks=1` :** Leader writes locally and returns ACK. Risk of data loss if leader crashes before follower fetches.
- **`acks=all` (-1):** Leader waits until **all active ISR members** acknowledge the append.
- **`min.insync.replicas` :** Enforces the minimum number of healthy ISR nodes required to satisfy an `acks=all`

write.
- Recommended enterprise standard: `replication.factor=3` + `min.insync.replicas=2` + `acks=all`. This tolerates 1 broker failure with **zero data loss** while maintaining write availability.
- If live ISR drops below `min.insync.replicas`, broker rejects writes with `NotEnoughReplicasException`.

Unclean Leader Election (`unclean.leader.election.enable`)

- `false` (Enterprise Default): If all ISR nodes crash, partition stays offline until an ISR member reboots. Zero data loss.
- `true`: Allows an out-of-sync follower to become leader. Re-enables writes immediately, but permanently discards un-replicated records.

---

6. KRaft (Kafka Raft Metadata Mode)

KRaft eliminates ZooKeeper by managing cluster metadata directly inside Kafka:
- Replaces external ZooKeeper with an internal Raft consensus quorum of Controllers.
- Metadata is stored in a special internal partition (`@metadata`) and replicated across controller nodes.
- **Deployment Modes:**
- **Combined Mode:** Single JVM acts as both broker and controller (`process.roles=broker,controller`). Ideal for labs and development.
- **Isolated Mode:** Dedicated controller nodes separate from broker data nodes (`process.roles=controller`). Mandatory for production.

---

7. Producer Internals, Batching & Idempotence

Dual-Threaded Architecture

1. **Main Thread:** Serializes key/value, computes target partition via MurmurHash2, and appends the message into the `RecordAccumulator`.
2. `RecordAccumulator`: Manages memory pools (`buffer.memory=32MB`). Allocates `ProducerBatch` buffers (sized by `batch.size`, default 16KB/32KB) per partition.
3. **Sender Thread:** Asynchronously monitors batches. Triggers transmission when:
   - A batch reaches `batch.size`, OR
   - Wait time exceeds `linger.ms` (e.g. 10ms–20ms), OR
   - Memory pressure forces a buffer flush.

Idempotent Deduplication (`enable.idempotence=true`)

- Default from Kafka 3.0+.

- Broker assigns a 64-bit **Producer ID (PID)** upon handshake.
- Producer assigns a monotonic **Sequence Number** ($0, 1, 2\dots$) to each record per partition.
- If an ACK drops in the network and the producer retries, the broker detects the duplicate sequence number, discards the duplicate record, and re-ACKs the producer cleanly.
- Preserves ordering even with `max.in.flight.requests.per.connection=5`.

Compression Codecs

Compression is applied per batch by the producer and decompressed by the consumer:
- **Snappy:** Low CPU overhead, high throughput (industry default for general streaming).
- **LZ4:** Very fast compression/decompression, low latency.
- **Zstandard (zstd):** High compression ratio with tunable speed levels.
- **GZIP:** Maximum compression ratio, but heavy CPU penalty.

---

8. Consumer Groups, Rebalancing & Resiliency

Consumer Groups

- Consumers sharing a `group.id` divide partitions among themselves.
- **Strict Limit:** Exactly 1 consumer per partition in the same group. Extra consumers sit idle.

Cooperative Sticky Rebalance Protocol

- **Old Eager Rebalance:** Revoked all partitions from all workers ("Stop-the-World"), causing massive consumer lag spikes.
- **Cooperative Sticky Rebalance (`CooperativeStickyAssignor`):** Workers continue reading non-affected partitions while only migrated partitions are revoked and reassigned.

Static Membership (`group.instance.id`)

Assigning a persistent `group.instance.id` allows rolling restarts and deployments without triggering unnecessary rebalances, provided the instance restarts within `session.timeout.ms`.

Liveness Timeouts

- `heartbeat.interval.ms` (default 3s): Frequency of keep-alive heartbeats to the group coordinator.
- `session.timeout.ms` (default 45s): Deadline before the broker evicts a silent consumer and triggers rebalance.
- `max.poll.interval.ms` (default 300s / 5 min): Maximum time spent inside record processing logic. If processing takes longer, the consumer is evicted to prevent processing stalls.

- `max.poll.records` (default 500): Number of records returned per poll. Lower this if record processing is CPU-heavy.

Dead Letter Topics (DLT)

To prevent malformed payloads (poison pills) from causing infinite consumer crash loops:
1. Wrap message deserialization in `try/except`.
2. Forward corrupt records to `<topic>.DLT` with error metadata headers.
3. Commit the partition offset and continue processing subsequent records.

---

9. Delivery Semantics Matrix

Guarantee	Mechanism	Failure Behavior	Production Fit
At-Most-Once	Auto-commit offset *before* processing finishes	Crashes cause lost messages	High-volume metrics, loss tolerable
At-Least-Once	Manual offset commit *after* processing finishes	Crashes cause message replays	Enterprise default; requires idempotent sinks
Exactly-Once (EOS)	`transactional.id` + `read_committed` + idempotent producer	Atomic produce and offset commit	Financial transactions, Kafka-to-Kafka pipelines

---

10. Operations, Monitoring & Golden Signals

Monitor these core JMX MBeans:
- `OfflinePartitionsCount` **(Target: 0):** Partitions with no active leader. Page immediately!
- `UnderMinIsrPartitionCount` **(Target: 0):** Partitions where active ISR < `min.insync.replicas`. `acks=all` writes are failing!
- `UnderReplicatedPartitions` **(Target: 0):** Partitions where followers lag behind leader.
- `ActiveControllerCount` **(Target: Exactly 1):** Sum across the cluster must equal 1. More than 1 indicates split-brain; 0 indicates cluster paralysis.
- **Consumer Lag:** $\text{LogEndOffset} - \text{CurrentOffset}$. Increasing lag indicates downstream bottlenecks.